There is no Leech tree on 18 vertices

and related distinct-distance tree problems

Lingsen Meng

Department of Earth, Planetary, and Space Sciences,
University of California, Los Angeles, CA 90095, USA
meng@epss.ucla.edu [TODO: author to confirm e-mail address]

Draft of 19 August 2026

Abstract

A *Leech tree* of order n is a tree on n vertices with positive integer edge weights whose $\binom{n}{2}$ pairwise path-weights are exactly $1, 2, \ldots, \binom{n}{2}$. Leech (1975) found five such trees, of orders $2, 3, 4, 4, 6$; Taylor (1977) showed that the order must be a square or a square plus two; computer searches by Székely, Wang and Zhang (2005) and by Calhoun, Ferland, Lister and Polhill (2007) excluded the admissible orders 9, 11 and 16, so that $n = 18$ has been the smallest open order. We show that there is no Leech tree on 18 vertices. The proof is an exhaustive computer search over "forced forests" in the sense of Calhoun et al., organised so that every weighted forest that can occur as the set of lightest edges of a Leech tree is generated exactly once up to isomorphism. The search tree has 59,779,854,336 nodes and contains no tree on 18 vertices; its per-level node counts are a mathematical invariant (the number of non-isomorphic forced forests with a given number of edges) which we report and which was reproduced bit-for-bit by three independent runs on two architectures and two compilers, and by an independent clean-room implementation written from the algorithm description alone. We also record the same statistics for $n \le 17$, where the search reproduces the known results, and discuss what would be needed to turn the computation into a formally checkable certificate. Variants of the same two engines settle several neighbouring questions: for the minimal distinct distance trees of Calhoun et al. we obtain $M(11) = 60$, with exactly two minimal trees, and $77 \le M(12) \le 78$ (their bounds were $59 \le M(11) \le 77$, $69 \le M(12) \le 94$); there is no modular Leech tree of order 9 or of order 11, while modular Leech trees of order 5 do exist, contrary to a statement quoted in the literature; and there are exactly six leaf-Leech trees with six leaves, five of which contain an irreducible vertex, which answers Question 3.1 of Ozen, Wang and Yalman in the affirmative.

Keywords. Leech tree, perfect distance tree, distinct distance tree, minimal distinct distance tree, modular Leech tree, leaf-Leech tree, Golomb ruler, Sidon set, exhaustive search, computer-assisted proof.
MSC 2020. 05C05, 05C78, 05C85, 11B83.

1 Introduction

Let T be a tree on n vertices with a weight $w(e) \in \mathbb{Z}_{>0}$ on every edge, and let $d(x, y)$ denote the weight of the (unique) x–y path. Following Leech [11] we call (T, w) a *Leech tree* (also *perfect distance tree*, *perfect distance-distinct tree*) if the multiset $\{d(x, y) : \{x, y\} \subseteq V(T)\}$ equals $\{1, 2, \ldots, N\}$ with $N = \binom{n}{2}$. Leech exhibited the five Leech trees of Table 1 and asked whether there are any others. None has been found since.

Necessary conditions on the order were given by Taylor [19]: counting odd distances shows that a Leech tree of order n exists only if $n = k^2$ or $n = k^2 + 2$ for some integer k (Lemma 2.2

Table 1: The five known Leech trees [11]. Weights are listed as edge lists u–v:w on vertices $0, 1, \ldots$; each labelling is unique up to isomorphism.

n	N	tree	weighted edges	remark
2	1	K_2	0–1:1	
3	3	P_3	0–1:1, 1–2:2	
4	6	$K_{1,3}$ (star)	0–1:1, 0–2:2, 0–3:4	
4	6	P_4 (path)	0–1:1, 1–2:3, 2–3:2	weight 3 must be the middle edge
6	15	$B_{2,2}$ (double star)	0–1:1, 0–2:2, 0–3:5, 3–4:4, 3–5:8	centres $0, 3$ joined by weight 5

below). The admissible orders are therefore $2, 3, 4, 6, 9, 11, 16, 18, 25, 27, \ldots$. Székely, Wang and Zhang [18] excluded $n = 9$ (also excluded by Shen Lin's search reported by Taylor [20]) and $n = 11$ by computer search, and proved asymptotic upper bounds on the length of a path and on the maximum degree of a Leech tree. Calhoun, Ferland, Lister and Polhill [3] introduced *distinct distance trees* (all path weights distinct, not necessarily forming an interval), gave a backtracking algorithm over weighted forests with a *forced next weight* (their Lemma 2.6 and Algorithm 2.7), and determined all perfect distance trees of order $n < 18$; in particular they excluded $n = 16$. Their summary states the bound as $n \leq 24$, but their abstract, their Table 1 (where the entry for $n = 18$ is only the trivial lower bound), and every later paper give $n < 18$; we treat "24" as a misprint. Further structural results—the non-existence of Leech trees of diameter 3 for $n \geq 7$, of certain "brooms", the "leaf-Leech" transformation, and Luo and Yu's finiteness result for diameter 4 and improved degree bound—are in [12, 15, 21]; a Leech labelling of general graphs was introduced in [8]. The order $n = 18$ is listed as the smallest open case, for instance in [15, 21] and in the FrontierMath open-problem list [6].

In this note we settle the case $n = 18$.

Theorem 1.1. *There is no Leech tree on* 18 *vertices.*

The proof is a computer search. Its mathematical content is small—the forcing lemma of [3, 18] and an elementary description of the automorphism group of a forest with distinct edge weights (Lemma 3.2), which yields a search tree in which every relevant weighted forest appears exactly once up to isomorphism—and its cost is modest (about 6×10^{10} search nodes, a few CPU-hours). We nevertheless think that a careful account is warranted, for three reasons. First, because $n = 18$ has been repeatedly cited as open, the result should be stated with a precise description of what was computed and how it was checked. Second, the per-level node counts of the search are a well-defined combinatorial quantity (the number of non-isomorphic "forced forests" with k edges that fit inside a Leech tree of order 18), so that the computation is falsifiable in a much stronger sense than a bare "no solution found": any re-implementation must reproduce eighteen integers, not one bit. Third, we want to be explicit about the status of the result as a computer-assisted theorem, including the parts that are and are not covered by independent verification (Section 7).

The rest of the note is organised as follows. Section 2 collects the lemmas that are used, marking which are classical and which are (minor) additions. Section 3 describes the algorithm and proves that it is complete and free of isomorphic duplicates. Section 4 describes the verification strategy. Section 5 states the results for $n \leq 18$, including the per-level tables. Section 6 reports what the same two engines give on three neighbouring problems—minimal distinct distance trees, modular Leech trees, and leaf-Leech trees—where the literature leaves gaps that a search of this kind can close. Section 7 discusses the status of the proof and what remains open.

2 Preliminaries

Throughout, n is the order, $N = \binom{n}{2}$, and "distance" means weighted path length. Weighted forests are considered up to isomorphism (a bijection of vertices preserving edges and weights). We say that a weighted forest is *distance-distinct* if the distances between pairs of vertices in the same component are pairwise distinct. All lemmas below are elementary; we include proofs of the two that carry the main argument (Lemmas 2.1 and 3.2) and sketch the rest.

Lemma 2.1 (forcing lemma; [18, "Find-Next-Weight"], [3, Lemma 2.6])**.** *Let (T, w) be a Leech tree with edge weights $w_1 < w_2 < \cdots < w_{n-1}$, and for $1 \le k \le n-1$ let F_{k-1} be the forest formed by the edges of weights $w_1, \ldots, w_{k-1}$. Then w_k equals the least positive integer that is not a distance within a component of F_{k-1}.*

Proof. Let m be that least missing integer. If $w_k < m$ then w_k is already a distance in F_{k-1}, contradicting distinctness. If $w_k > m$ then m is not an edge weight, and any path realising m has at least two edges, hence every edge on it has weight $< m < w_k$ and lies in F_{k-1}; so m would be a distance in F_{k-1}, contradiction. Hence $w_k = m$. □

Consequently $w_1 = 1$, $w_2 = 2$, and $w_3 \in \{3, 4\}$ according as the edges of weight 1 and 2 are non-adjacent or adjacent; a Leech labelling of a given tree is determined by the *order* in which its edges receive their weights. Note that Lemma 2.1 fails for graphs with cycles [3].

Lemma 2.2 (Taylor's parity condition [19]; cf. [18, Thm. 1], [3, Thm. 1.3])**.** *If a Leech tree of order n exists, then $n = k^2$ or $n = k^2 + 2$ for an integer k; the vertex classes A, B at even and odd weighted distance from a fixed vertex satisfy $|A|\,|B| = \lceil N/2 \rceil$. For $n = 18$: $N = 153$, $\{|A|, |B|\} = \{11, 7\}$.*

Sketch. $d(x, y) \equiv d(x, z) + d(y, z) \pmod 2$ in a tree; a distance is odd iff its ends lie in different classes, so $|A|\,|B| = \lceil N/2 \rceil$, and $(|A| - |B|)^2 = n^2 - 4|A|\,|B| \in \{n, n-2\}$. □

Lemma 2.3 (cut identity)**.** *For any weighted tree, $\sum_{x<y} d(x, y) = \sum_e c_e\, w(e)$, where $c_e = s_e(n - s_e)$ is the number of vertex pairs separated by e and s_e is the number of vertices on one side of e. For a Leech tree the left side is $N(N+1)/2$.*

Lemma 2.4 (Golomb and containment bounds)**.** *In a distance-distinct weighted tree:*

(a) the vertices of any path with h edges, placed on a line at their cumulative distances, form a Golomb ruler with $h+1$ marks, so $d(x, y) \ge G(h+1)$ where $G(m)$ is the length of an optimal Golomb ruler with m marks;

(b) if the tree is Leech, every path weight is at most N, so the (hop) diameter D satisfies $G(D+1) \le N$;

(c) (containment) if s_x, s_y denote the number of vertices "behind" x and y on the x–y path (so that $s_x s_y$ pairs have paths containing the x–y path), then in a Leech tree $d(x, y) \le N + 1 - s_x s_y$; in particular $w(e) \le N + 1 - c_e$ for every edge.

The values

$$G(1), \ldots, G(17) = 0, 1, 3, 6, 11, 17, 25, 34, 44, 55, 72, 85, 106, 127, 151, 177, 199$$

(OEIS A003022 [14]) are used; $G(m)$ for $m \le 13$ was re-verified by exhaustive search for this project, and $G(14), \ldots, G(17)$ are taken from the literature (Shearer's tables [17]; the proof of the 19-mark ruler [5]; the distributed.net OGR project [4]). Since $G(16) = 177 > 153 = N$, a Leech tree of order 18 has hop-diameter at most 14. This strengthens the exact finite form of [18, Thm. 2] (which gives diameter ≤ 15 at $n = 18$).

Lemma 2.5 (star-Sidon degree bound)**.** *Let v be a vertex of degree d in a Leech tree of order n with incident weights $a_1 < \cdots < a_d$. Then the a_i and the $\binom{d}{2}$ sums $a_i + a_j$ $(i < j)$ are $d + \binom{d}{2}$ pairwise distinct integers in $[1, N]$; call a set of positive integers with this property (pairwise sums of* distinct *elements distinct and different from the elements)* star-Sidon*. Let $S(d)$ be the minimum of $a_{d-1} + a_d$ over star-Sidon sets of size d; then $S(d) \le a_{d-1} + a_d \le N + 1 - b_{(1)}b_{(2)}$, where $b_{(1)} \le b_{(2)}$ are the two smallest branch sizes at v. Exhaustive computation gives*

d	2	3	4	5	6	7	8	9	10	11	12
$S(d)$	3	6	11	19	31	43	63	80	110	138	169

and S is non-decreasing in d. Since $S(12) = 169 > 153$, the maximum degree of a Leech tree of order 18 is at most 11 (a degree-11 vertex is possible only with $b_{(1)}b_{(2)} \le 16$).

Remark (correction). An earlier version of this lemma asserted that the incident weights form a Golomb ruler (equivalently a Sidon set in the strong sense) and used $G(d) + G(d-1) + 2$ in place of $S(d)$. That is false: the Leech condition only makes $\{a_i\}$ a *weak* Sidon set (sums of two distinct elements distinct); three-term arithmetic progressions among the a_i are not excluded. The star $K_{1,7}$ with weights $\{1, 2, 4, 8, 14, 19, 24\}$ has all 28 distances distinct, yet $a_6 + a_7 = 43 < 44 = G(7) + G(6) + 2$; the correct minima are those in the table (the two sequences differ from $d = 6$ on). The counterexample was found during the internal audit of the search code (Section 4). The maximum-degree consequence at $n = 18$ ($\Delta \le 11$) is unaffected: it also follows from an exhaustive search over star-Sidon sets with all elements and sums ≤ 153, which finds a set of size 11 (e.g. $\{1, 2, 4, 7, 12, 23, 32, 41, 56, 70, 83\}$) and none of size 12. Lemma 2.5 is a finite, explicit version of the idea behind [18, Thm. 3] and, we believe, of [12]; we could not access the text of [12] and make no claim of priority. **[TODO: obtain Luo–Yu 2024 and compare with Lemma 2.5]**

Lemma 2.6 (weight bounds; [3, Thms. 2.8, 2.9])**.** *In a Leech tree of order n the largest edge weight satisfies $m_1 \le \lfloor (4n-1)^2/48 \rfloor$ and the second largest $m_2 \le \lfloor (N-1)/2 \rfloor$; more generally the weights of any set of edges lying on a common path sum to at most N. At $n = 18$: $m_1 \le 105$, $m_2 \le 76$.*

What is used where. It is worth being precise about which of these statements the proof of Theorem 1.1 depends on. The forest search of Section 3 uses only Lemma 2.1, the trivial facts that all distances are distinct and at most N and that a subforest of a Leech tree has at most n vertices, the "common path" case of Lemma 2.6 (any two edges lie on a common path, so $w(e) + w(f) \le N$; this rule was found never to prune anything at $n \le 18$), and Lemma 3.2 below. Lemmas 2.2–2.5 and the rest of Lemma 2.6 are *not* needed for Theorem 1.1; they are used in the independent per-topology engine (Section 4.2) as pruning rules and as topology filters, and are recorded here because they were verified in the course of the project and because Lemma 2.5 corrects a statement that circulated in our own notes. Lemmas 2.1, 2.2, 2.6 and Lemma 2.4(a),(b) are from the literature (2.4(b) as applied to the diameter is folklore; [3, Prop. 1.2] uses it for paths); Lemma 2.3, Lemma 2.4(c), Lemma 2.5 (in the corrected form) and Lemma 3.2 do not seem to appear in the Leech tree literature, though none of them is deep.

3 The algorithm

3.1 Forced forests

Fix n and $N = \binom{n}{2}$. By Lemma 2.1, if (T, w) is a Leech tree and $t \ge 1$, then the subforest $F_t = \{e \in T : w(e) < t\}$ satisfies: (i) F_t is distance-distinct with all distances in $[1, N]$; (ii) F_t

has at most n vertices (we regard F_t as a forest without isolated vertices; the vertex set is the set of endpoints of its edges); (iii) if t' is the least positive integer that is not a distance of F_t, then either $F_t = T$ (in which case $t' = N + 1$), or T has an edge e of weight exactly t' and $F_{t'+1} = F_t + e$. Call a weighted forest *forced* (for the target $[1, N]$ and order n) if it arises from the empty forest by repeatedly adding an edge whose weight is the least missing distance, in one of three ways: *join* (the new edge connects two existing components), *attach* (one endpoint is a new vertex), or *new* (both endpoints are new; a fresh single-edge component), never exceeding n vertices and never repeating a distance or exceeding N. This is exactly the search of [3, Alg. 2.7]. By (iii) every Leech tree of order n is a forced forest with $n - 1$ edges and n vertices; conversely a forced forest with n vertices and $n - 1$ edges is a tree in which every value $1, \ldots, N$ is realised (the least missing value has been pushed to $N + 1$), hence a Leech tree.

The natural DFS enumerates forced forests level by level (level $k = k$ edges). Its cost is dominated by the last few levels, and without symmetry reduction the same abstract forest is generated many times, since the endpoints of single-edge components and the components themselves can be listed in many orders. The one idea that makes the search cheap is complete isomorph rejection, for which the following two lemmas suffice.

3.2 Isomorph rejection

Lemma 3.1 (canonical parent)**.** *In a forced forest the edge weights are pairwise distinct and the weights of successive edges are strictly increasing; hence removing the edge of maximum weight from a forced forest with $k+1$ edges yields a forced forest with k edges, and this parent is well defined up to isomorphism.*

Proof. Each new weight t is the least missing distance and becomes a distance once the edge is added, so later weights are larger. Deleting the last edge returns to the previous state of the DFS. □

Lemma 3.2 (automorphisms of a distinct-weight forest)**.** *Let P be a forest without isolated vertices whose edge weights are pairwise distinct. Then the group of weight-preserving automorphisms of P is generated by the transpositions of the two endpoints of the single-edge components of P; in particular $\mathrm{Aut}(P) \cong \mathbb{Z}_2^s$, where s is the number of single-edge components, and it acts on components independently.*

Proof. Let φ be a weight-preserving automorphism. Since the weights are distinct, φ fixes every edge setwise. If two distinct edges e, f share a vertex v, then $\varphi(v) \in e \cap f = \{v\}$, so v is fixed. In a component with at least two edges every edge has an endpoint of degree ≥ 2; that endpoint is fixed, and then the other endpoint of the (setwise fixed) edge is fixed too. So every component with ≥ 2 edges is fixed pointwise. A single-edge component admits exactly the identity and the endpoint swap, and swaps of different components commute and are independent. □

Proposition 3.3 (exactly-once generation)**.** *Consider the following restricted DFS. Vertices are labelled in order of appearance. At a node P (a labelled forced forest), the* eligible *vertices are all vertices of P except the larger-labelled endpoint of each single-edge component. Children are: all joins between eligible vertices $x < y$ in different components; all attachments of a new vertex to an eligible vertex x; and, if P has at most $n - 2$ vertices, the single "new" child. (Each child is kept only if it is a forced forest, i.e. passes the distance and vertex tests.) Then every abstract forced forest with k edges is generated exactly once at level k, for every k.*

Proof. Induction on k. Assume the level-k nodes are pairwise non-isomorphic and cover every abstract forced forest with k edges. Let G be an abstract forced forest with $k + 1$ edges, e its

maximum-weight edge and $P = G - e$; by Lemma 3.1, P is a forced forest with k edges, so it appears exactly once as a labelled node P'. Two extensions $P' + (x, y, t)$ and $P' + (x', y', t)$ (with the same forced weight t) are isomorphic iff some $\varphi \in \mathrm{Aut}(P')$ maps $\{x, y\}$ to $\{x', y'\}$: an isomorphism of the children fixes the unique maximum-weight edge and restricts to an automorphism of the parents. By Lemma 3.2, $\mathrm{Aut}(P')$ is the product of the endpoint swaps of the single-edge components, so the orbit of an endpoint set is obtained by independently swapping the endpoint(s) that lie in single-edge components. Choosing the smaller label in each single-edge component therefore selects exactly one representative per orbit for joins (unordered pairs in distinct components), for attachments (orbit of x), and trivially for the "new" child. Children of non-isomorphic parents are non-isomorphic (their canonical parents would be isomorphic). □

Proposition 3.3 is what allows the search tree to be *counted*: its level-k node count equals the number of non-isomorphic forced forests with k edges for the given (n, N). This makes the computation checkable in the strong sense discussed in the Introduction. Because Aut is recomputed from the current component structure at each node, no automorphism state is cached; the appearance-order labelling is used only to pick "the smaller endpoint of a two-vertex component".

3.3 Implementation and sharding

The engine (`forest_search.cpp`, C++17, no dependencies) keeps, for each vertex x, the bitset $D_0[x]$ of distances from x within its component, and the bitset R of realised distances. Adding an edge (x, y, t) generates the new distances as shifted unions $D_0[x] + t + D_0[y]$; a child is rejected as soon as a shifted set meets R or exceeds N, and a per-vertex prefilter discards x when already the distances from x to the *new neighbour* alone collide (these are a subset of the new distances of any join or attachment at x). All state is undone on return (incremental, no copies). The only other prune is $t + w_{\max} \le N$ (Lemma 2.6, common path), which never fires at $n \le 18$.[1] There are no bounds, counting arguments or lookahead in this engine; its efficiency comes from Proposition 3.3 (which alone reduced the node count roughly tenfold at $n = 12$–13 relative to the same DFS without isomorph rejection) and from bit-parallel distance sets.

For parallelism the DFS is sharded at a fixed level L: the level-L nodes are enumerated in DFS order (deterministic), and shard i of K explores the subtrees of the level-L nodes with index $\equiv i \pmod K$; the levels $\le L$ are visited by every shard. Hence, if $h_i(d)$ is the number of level-d nodes visited by shard i, one must have $h_i(d) = h(d)$ for $d \le L$ and every i, and $\sum_i h_i(d) = h(d)$ for $d > L$; the total number of visited nodes is $\sum_i \mathrm{nodes}_i = \mathrm{unique} + (K - 1) \cdot \mathrm{prefix}(\le L)$. These identities were checked in 36 $(n, \mathrm{target}, K, L)$ configurations, including K not dividing the number of level-L nodes and K larger than it. The verdict script for a sharded run asserts: exactly K records, indices $0, \ldots, K - 1$ each once, identical (K, L), all `DONE`, and the histogram identities above.

4 Verification

The engine of Section 3 was written by an AI coding agent under human direction (see the statement in Section 8). We therefore treated it as untrusted and audited it in the following ways. Everything in this section refers to code that is available with the paper (Section 9).

[1] More precisely, the clean-room implementation of Section 4.5 found that this rule fires exactly once at $n = 12$ (removing one level-10 node) and never in the $n = 18$ tree; since it is a necessary condition it cannot remove a solution, and both engines apply it, so the per-level counts reported here are those *with* the rule.

4.1 Independent enumerator (oracle) and differential tests

A Python enumerator (`referee_forest_enum.py`), written independently of the C++ code from the definition in Section 3.1, generates *all* labelled join/attach/new children of every node without any symmetry rule, keeps a child iff its full distance multiset (recomputed from scratch by BFS) is duplicate-free and inside the target, and de-duplicates each level by a complete canonical form for distinct-weight forests without isolated vertices (the sorted multiset of per-vertex sorted incident-weight tuples). Its per-level counts equal the engine's for every level and every $n \leq 12$ (at $n = 12$: $1, 1, 2, 8, 41, 229, 1384, 8877, 58933, 278539, 356479$ nodes on levels $0, \ldots, 10$, 23 minutes in Python), for planted targets with holes at $n = 6, \ldots, 9$, and for the prefix of the $n = 17$ and $n = 18$ trees through level 9 (at $n = 18$: 480,085 level-9 forests). A second Python mode that re-implements the engine's own generation rule *without* de-duplication produced zero canonical-form collisions in all these cases, i.e. the isomorph rejection is irredundant as implemented and not only as proved.

Two independent formulations of the same problem were used as further oracles: (a) a per-topology engine (Section 4.2) whose solution counts, summed over *all* unlabelled trees of a given order, must equal the forest engine's; this was checked on 45 instances ($n = 6, \ldots, 12$, standard and planted targets, including targets with two non-isomorphic solutions) with 0 mismatches; and (b) a plain edge-by-edge backtracking over target values with *no* forcing lemma, summed over topologies as labellings divided by $|\operatorname{Aut}|$, for $n \leq 9$. Full-depth planted instances at $n = 13, \ldots, 18$ (random weighted trees with distinct distances, custom target set) were run through the engine: in all 16 instances the planted tree was found among the solutions and every reported solution was re-checked; these are the only tests that exercise the $n = 18$, full-vertex-count code path with a witness. Finally the production binary was shown to be a faithful build of the archived source by reproducing its depth histograms at $n = 4, \ldots, 14$ and the exact node count (100,024,625) of one $n = 18$ production shard from a byte-for-byte rebuild.

4.2 Independent method: per-topology search

Before the forest engine we built a per-topology exact search (`leech_search.cpp`): for each of the 123,867 unlabelled trees on 18 vertices (OEIS A000055 [14]; enumerated by two independent generators, the WROM algorithm [23] as implemented in NetworkX [7] and nauty's `gentreeg` [13], which agree for $n = 5, 9, 11, 16, 18$: $3, 47, 235, 19320, 123867$) it branches on which unassigned edge receives the least missing value (Lemma 2.1) and prunes with distinctness, the containment and Golomb windows (Lemma 2.4), the cut identity (Lemma 2.3), Hall-type counting on value windows, Taylor's parity (Lemma 2.2), Lemma 2.6, and complete symmetry breaking over Aut of the topology (verified as *exactly one representative per orbit* by the identity $\text{count(sym)} \cdot |\operatorname{Aut}| = \text{count(nosym)}$ on planted instances with $|\operatorname{Aut}|$ up to 20,000). This engine was audited by a differential harness against a rule-free DFS and against a CP-SAT enumeration [16]: 574 instances (all topologies $n = 7, \ldots, 10$ with planted targets, the standard target $n = 7, \ldots, 9$, parity-skewed families, symmetry stress set) $\times$ 3 binaries $\times$ 32 flag subsets, and a second seed with 560 instances, with 0 discrepancies. It reproduces the literature id-by-id at $n = 9$ (47 topologies) and $n = 11$ (235), agreeing with CP-SAT and with the DRAT-certified SAT route (Section 4.3), and it has now decided *all* 19,320 topologies of order 16 on the cluster as UNSAT (19,308 with the first-generation engine at a 1800 s cap; the 12 topologies that hit the cap were re-run with the improved engine and finished in 673–1317 s each). This is an independent-method reproduction of the non-existence of a Leech tree of order 16 [3].

The per-topology engine is between three and four orders of magnitude slower in total than the forest engine (it re-derives the same small-weight forests once per topology: $n = 16$

took 321 CPU-s with the forest engine against roughly 500–700 CPU-h topology-by-topology), which is why it is used as a *cross-check* rather than as the primary proof.

At $n = 18$ this cross-check is *incomplete*, and we state its status precisely. A first pass over all 122,344 surviving topologies (those left after the proven filters of Section 5.2) has finished on the cluster with a 300 s time cap per topology: 73,021 topologies UNSAT, 49,323 undecided at the cap, and—this is the part that matters—0 topologies SAT. The 49,323 undecided topologies are being re-run with the improved engine at a 3600 s cap; that run had not finished at the time of writing. The independent-method cross-check therefore currently covers 73,021 of the 122,344 survivor topologies, i.e. 59.7% of them; the remaining 1,523 of the 123,867 topologies had been removed beforehand by the filters of Section 5.2, and the forest engine of Theorem 1.1 uses no topology filters at all. No topology anywhere in the project, at any order, has ever been reported SAT except for those carrying the three known Leech trees of orders 4 and 6. Until the re-run is complete, Theorem 1.1 rests on the forest engine, its audits (Section 4.1), the three bit-identical replications (Section 4.4) and the clean-room implementation (Section 4.5); the per-topology run is corroboration by a different method, not a second complete proof. **[TODO: report the completed per-topology outcome for $n = 18$ once the re-run of the 49,323 undecided topologies has finished; if it disagrees with the forest engine, the theorem must be withheld.]**

4.3 SAT encoding with checked proofs ($n \leq 11$)

For small orders we also produced machine-checkable certificates: an order-encoding CNF per topology (pair/value variables, ternary sum relations along a rooted tree, sibling-subtree symmetry breaking), solved with CaDiCaL [1, 2] (version 3.0.1; Kissat 4.0.4 was used as a second solver) producing DRAT proofs, checked with `drat-trim` [22]. All 47 topologies at $n = 9$ and all 235 at $n = 11$ are UNSAT with verified proofs (also the 5 non-Leech topologies at $n = 6$; the sixth yields the known tree); this reproduces [18] with independent certificates. The approach does not scale (30 random $n = 16$ topologies: 0/30 decided in 600 s each), so no DRAT-style certificate exists for $n = 18$; see Section 7.

4.4 Replication of the $n = 18$ run

The $n = 18$ forest search was run three times with the reference engine, with two shard layouts, on two architectures and with two compilers, and once more with the clean-room implementation of Section 4.5; the per-shard depth histograms were compared (Table 2).

Table 2: Replication of the $n = 18$ forest search. $\sum$nodes counts the shared prefix (levels $\leq L$) once per shard; "unique" $= \sum$nodes $- (K-1) \cdot 73{,}408$, where 73,408 is the number of nodes on levels $0, \ldots, 8$. Hoffman2 is the UCLA shared cluster (SLURM array jobs); "local" is an Apple M4 laptop.

run	machine / compiler	(K, L)	$\sum$nodes	unique nodes	levels 9–16	L17	trees
Hoffman2 job 83703	x86_64, gcc 11.5	(210, 8)	59,795,196,608	59,779,854,336	Table 4	0	0
Hoffman2 job 83752	x86_64, gcc 11.5	(175, 9)	59,876,162,118	59,779,854,336	identical	0	0
local, reference	arm64 M4, clang	(512, 8)	59,817,365,824	59,779,854,336	n/a[2]	—	0
local, clean-room	arm64 M4, clang	(100, 8)	59,787,121,728	59,779,854,336	identical	0	0

The four sums of visited nodes differ exactly by the multiplicity of the shared prefix ($(K-1) \cdot \text{prefix}(\leq L)$), and the unique node counts agree bit-for-bit. For the two cluster runs and

[2]The local driver of the reference engine discarded the per-shard stderr histograms; only the node counts and status of each shard were kept. The identity of the unique node count with the two cluster runs is the check available for this run.

the clean-room run the per-level sums agree with each other on every level, and agree with the Python oracle on levels ≤ 9. The node counts of the reference engine are deterministic, so its three-fold replication detects hardware, compiler and job-management faults (lost or truncated shards, mis-sharding), not algorithmic errors; the algorithmic checks are those of Sections 4.1–4.2 and 4.5.

4.5 Clean-room re-implementation

A second implementation of the forced-forest search (`cr_forest.c`, C, single file) was written from the algorithmic description of Section 3 and [3] only, by an agent that had no access to the production source; it uses its own state representation (state copied per DFS level, full within-component distance matrix, component identifiers by smallest label) and its own ordering of children, and shares with the reference engine only the mathematical rules of Proposition 3.3. It comes with its own brute-force Python oracle (all labelled children, BFS re-computation of distances, canonical-form de-duplication per level), with which it agrees at every level for $n = 4, \ldots, 12$, and it passes the same sharding identities on 11 (n, K, L) cases. It reproduces the reference engine's per-level counts at every level for $n = 4, \ldots, 16$ (at $n = 16$: 1,096,039,152 nodes, identical histogram), for levels $0, \ldots, 12$ at $n = 17$, and—run over the full $n = 18$ tree with $K = 100$, $L = 8$ (34,207 CPU-s at low priority on the laptop)—all eighteen per-level counts of Table 4, the unique node count 59,779,854,336, and no tree on 18 vertices. The verdict script `cleanroom/verify_18.py` performs the shard-integrity checks of Section 3.3 and prints the comparison with the reference counts.

5 Results

5.1 Orders $n \leq 17$

The forest engine finds exactly the known Leech trees and no others: at $n = 4$ two (the star and the path), at $n = 6$ one (the double star), and none at $n = 5$ and $7 \leq n \leq 17$. (The condition of Lemma 2.2 is not imposed, so the non-admissible orders are searched as well and come out empty, as they must.) Node counts and CPU times (single core, Apple M4, under load) are given in Table 3.

Table 3: Forest search for $n \leq 17$ (target $[1, \binom{n}{2}]$). "Last levels" lists the node counts of the three deepest non-empty levels. For $n = 17$ the run was sharded ($K = 32$, $L = 8$); the unique node count subtracts the 31 repeated copies of the 73,408-node prefix.

n	nodes (unique)	CPU	Leech trees	last levels
11	123,309	0.05 s	0	L7 8,645; L8 43,855; L9 69,144
12	704,494	0.14 s	0	L8 58,933; L9 278,539; L10 356,479
13	4,178,290	0.9 s	0	L9 424,878; L10 1.86M; L11 1.82M
14	25,486,327	6.4 s	0	L10 3.25M; L11 13.0M; L12 8.66M
15	162,497,458	44 s	0	L11 26.2M; L12 93.5M; L13 38.4M
16	1,096,039,152	321 s	0	L12 220.0M; L13 683.5M; L14 154.7M
17	7,875,306,893	1.7 CPU-h	0	(histograms not retained)

The full level histogram at $n = 16$ (levels $0, \ldots, 15$) is

$$1,\ 1,\ 2,\ 8,\ 41,\ 229,\ 1384,\ 8899,\ 62843,\ 480048,\ 3968025,\ 33269745,\ 220001476,\ 683511193,\ 154735257,\ 0.$$

The counts at $n = 16$ and $n = 17$ reproduce the non-existence results of [3]; those at $n = 9, 11$ reproduce [18], and are also confirmed by the per-topology engine, CP-SAT, and (for $n = 9, 11$)

DRAT-checked SAT proofs. The per-level counts at low levels are the same for all n once n is large enough for the vertex bound not to bite $(1, 1, 2, 8, 41, 229, 1384, 8899, 62843, \dots)$, and grow by a factor of roughly 7.5 per level; the whole tree grows by a factor of 6.5–7 per unit of n.

5.2 Order 18

Theorem 5.1 (Theorem 1.1, quantitative form)**.** *For $n = 18$, $N = 153$, the search tree of Section 3 has the per-level node counts of Table 4, a total of* 59,779,854,336 *nodes, and no node at level* 17*. Consequently there is no Leech tree of order* 18*.*

Table 4: Number of non-isomorphic forced forests with k edges for $(n, N) = (18, 153)$. Verified identical in both cluster runs and in the clean-room implementation; levels ≤ 9 also by the Python oracle.

level k	0	1	2	3	4	5
forests	1	1	2	8	41	229
level k	6	7	8	9	10	11
forests	1,384	8,899	62,843	480,085	3,984,162	35,540,837
level k	12	13	14	15	16	17
forests	332,597,341	2,896,330,052	17,017,193,146	37,669,242,243	1,824,413,062	**0**

The tree dies at the last two levels: of the 1.8×10^9 distance-distinct forced forests with 16 edges (i.e. one edge short of a spanning tree on 18 vertices, or 17 edges on fewer vertices), none can be completed. The whole computation cost about 5–11 CPU-hours depending on machine and load (roughly 0.3 μs per node unloaded; 39,708 CPU-s for the local 512-shard run at low priority); on the cluster each of the 210 shard processes ran for about 1.5 minutes of wall time. For comparison, a per-topology search of the same order was projected at 10^4–10^5 CPU-hours.

We record for completeness that the published structural results prune very little at the topology level: of the 123,867 trees on 18 vertices, the diameter bound (Lemma 2.4(b)), the degree bound (Lemma 2.5), the exclusion of stars, double stars, paths and the broom of [21] remove 152, leaving 123,715; adding real relaxations (majorisation of the distance vector, projection bound) leaves 122,344. The forest search does not use topologies at all.

6 Neighbouring problems settled by the same engines

The forced-forest search of Section 3 and the per-topology search of Section 4.2 are not specific to the target $\{1, \dots, N\}$. With small modifications they decide three neighbouring problems on which the literature stops short of exact values, and on which the published bounds are, in each case, more than twenty years old or explicitly left open. All results in this section were obtained after the $n = 18$ computation, with copies of the engines kept separate from the production sources; every witness reported below was re-checked by an independent breadth-first-search checker that shares no code with any engine, and the ones printed here are small enough to be verified by hand.

6.1 Minimal distinct distance trees: $M(11) = 60$ and $77 \leq M(12) \leq 78$

Calhoun, Ferland, Lister and Polhill [3] define: "If all of these distances are distinct, we call the tree a distinct distance tree. Define the function $M(n)$ to be the smallest integer so that there exists a distinct distance tree with n vertices and maximum distance $M(n)$" [3, p. 33];

a *minimal distinct distance tree* is one attaining $M(n)$, and a Leech tree is exactly a distinct distance tree with $M(T) = \binom{n}{2}$. They determine $M(n)$ for $n \le 10$ $(0, 1, 3, 6, 11, 15, 22, 30, 39, 50)$ and give the bounds of their Table 1 for $11 \le n \le 18$, among them $59 \le M(11) \le 77$, $69 \le M(12) \le 94$ and $80 \le M(13) \le 119$, with the remark that "it will take too long for the algorithm to check higher numbers of gaps and find the exact value of $M(n)$" [3, p. 44].

The forest engine adapts to $M(n)$ by replacing the forcing lemma with a three-way decision at each value level t (all values $< t$ decided): either t is already a distance, or t is declared a *gap* (never a distance; at most $D - \binom{n}{2}$ gaps are available when the target maximum is D), or t is the weight of the next edge, added by join, attach or new as before. Every state (forest, t) is again reached by exactly one decision sequence, so the isomorph rejection of Proposition 3.3 carries over verbatim and the number of solutions at $D = M(n)$ is the number of minimal trees up to isomorphism. Feasibility is decided for increasing D; the run at $D = M(n) - 1$ is the exclusion. The engine reproduces all of Calhoun's values and, where they are stated, their counts of minimal trees ($n = 7$: 2; $n = 8$: 2; $n = 9$: 1; $n = 10$: 1, with the same weights $1, 2, 3, 4, 5, 11, 14, 18, 28$), at costs from 215 nodes ($n = 7$, $D = 21$) to 2.2×10^7 nodes ($n = 10$, $D = 49$).

Theorem 6.1. *$M(11) = 60$, and there are exactly two minimal distinct distance trees on 11 vertices, namely (edge lists u–v:w)*

0–1:1, 0–2:2, 3–4:4, 5–6:5, 5–7:6, 3–8:8, 3–5:9, 3–9:16, 2–7:26, 7–10:27,
0–1:1, 0–2:2, 3–4:4, 5–6:5, 5–7:6, 3–8:8, 3–5:9, 3–9:16, 7–10:26, 0–7:27,

whose 55 distances are $\{1, \ldots, 60\}$ minus $\{7, 10, 21, 36, 54\}$ and $\{7, 10, 21, 36, 56\}$ respectively. Moreover $77 \le M(12) \le 78$ and $M(13) \le 105$.

The computations behind Theorem 6.1 are as follows (all on the cluster unless stated; "UNSAT" means the exhaustive run at that D found no tree and all shards completed). At $n = 11$: $D = 59$ UNSAT (1.80×10^8 nodes locally, 1.83×10^8 in 7 shards on the cluster), $D = 60$ two trees (5.86×10^8 nodes, 111 CPU-s). For $D = 61, \ldots, 65$ the numbers of distinct distance trees with maximum distance exactly D are $0, 6, 3, 12, 32$. As an independent-method check, a per-topology backtracking program with no forcing lemma, no forest search and no isomorph rejection was run over all 235 topologies of order 11: at $D = 59$ it finds 0 labellings in 12,620,879,225 nodes and at $D = 60$ it finds 18 labellings in 15,106,490,676 nodes, with identical node counts and identical solution lists on two machines (arm64/clang and x86_64/gcc); the 18 labellings are exactly the two trees above counted with their automorphisms (6 and 12). The same check at $n = 10$ gives 0 labellings at $D = 49$ and 6 (one tree, $|\mathrm{Aut}| = 6$) at $D = 50$. At $n = 12$ the exhaustive runs at $D = 69, 70, \ldots, 76$ are all UNSAT, at costs 3.7×10^8, 1.5×10^9, 5.2×10^9, 1.6×10^{10}, 4.2×10^{10}, 1.0×10^{11}, 2.4×10^{11} and 5.1×10^{11} nodes, so $M(12) \ge 77$; and the witness-hunting mode found the tree

0–1:1, 0–2:2, 3–4:4, 5–6:5, 0–3:6, 5–7:13, 8–9:14, 4–8:15, 6–9:16, 8–10:17, 9–11:37

with all 66 distances distinct and maximum 78, so $M(12) \le 78$. The exhaustive run at $D = 77$ (expected $\sim 10^{12}$ nodes) is queued; a search for a witness at $D = 77$ was still running, without success, after 5×10^{10} nodes. **[TODO: state the exact value of $M(12)$ once the exhaustive $D = 77$ run completes]** Appending one leaf to the $n = 12$ witness with $D = 79$ gives a 13-vertex distinct distance tree of maximum distance 105, whence $M(13) \le 105$ (Calhoun: 119).

6.2 Modular Leech trees: none of order 9 or 11, but two of order 5

Leach and Walsh introduced the modular relaxation, and Leach [9] states it as: "Let T be a tree on n vertices and let $k = \binom{n}{2} + 1$. We say that T is a modular Leech tree if there exists

an edge weighting function $w : E(T) \to \mathbb{Z}_k$ such that each of the $\binom{n}{2}$ paths within T has a distinct weight from $1, 2, \ldots, \binom{n}{2}$ with the sums taken modulo $\binom{n}{2} + 1$" [9, p. 1]; equivalently w induces a bijection between the paths of T and $\mathbb{Z}_k \setminus \{0\}$. Every Leech tree is a modular Leech tree, so orders $2, 3, 4, 6$ occur. Leach [9, Thms. 1–2] shows that if $n \equiv 2, 3 \pmod 4$ then a modular Leech tree of order n forces $n = m^2 + 2$ (so none exist for $n = 7, 10, 14, 15, \ldots$), that multiplying a labelling by a unit of $\mathbb{Z}_k$ gives a labelling, and, by computer search, that there is exactly one modular Leech tree of order 8 (over $\mathbb{Z}_{29}$, unique up to group and graph isomorphism). Nothing is stated there about $n \geq 9$.

Our engine here is a per-topology depth-first search over the frozen topology lists, placing edges in breadth-first order from a maximum-degree root and maintaining the set of realised residues in a 128-bit word; solutions are counted as orbits under the unit action of $\mathbb{Z}_k$. Two implementations with different symmetry breaking (restriction of the first label versus a lexicographic-minimum test over the unit stabiliser, the latter with forward checking) were written and give identical orbit counts for $n = 4, \ldots, 9$; brute force over all $(k-1)^{n-1}$ labellings independently confirms $n = 4, 5, 6$; and at $n = 8$ the search reproduces Leach exactly, finding the single topology of his Figure 3 with one labelling up to the leaf swap, our labelling being 3 times his in $\mathbb{Z}_{29}$.

Table 5: Modular Leech trees of order n over $\mathbb{Z}_k$, $k = \binom{n}{2} + 1$. "Orbits" counts $\mathbb{Z}_k$-Leech labellings up to multiplication by a unit (graph automorphisms are not divided out). New results in bold.

n	k	topologies	trees / orbits	search nodes	remark
4	7	2	2 / 4	16–18	Leech's two trees
5	11	3	**2 / 4**	58–127	see the discussion below
6	16	6	2 / 34	1.7×10^3–2.2×10^4	the Leech tree and one other
7	22	11	0	3.1×10^4–4.6×10^5	agrees with [9, Thm. 2]
8	29	23	1 / 2	2.4×10^5–7.6×10^5	= [9]
9	37	47	**0**	6.5×10^6–1.8×10^7	new
10	46	106	0	3.6×10^8–1.4×10^{10}	consistent with [9, Thm. 1]
11	56	235	**0**	1.3×10^{10}	new; single implementation

Table 5 summarises the outcome. The two new non-existence results are $n = 9$ and $n = 11$; both orders are permitted by Leach's theorems ($9 = 3^2$, $11 = 3^2 + 2$), so they were open. The order-9 result is produced by both implementations; the order-11 result comes from the faster implementation only (the slower one did not finish), and we label it accordingly as a single-implementation computation.

The order-5 entry requires comment. Leach writes: "By Leach and Walsh [6] and Theorem 2 we know that none exist for 5 or 7" [9, p. 2], his [6] being [10]. His Theorems 1–2 exclude $n = 7$; they say nothing about $n = 5$ (as $5 \equiv 1 \pmod 4$), so the order-5 statement rests entirely on [10], which we were not able to obtain. Under the definition quoted above, modular Leech trees of order 5 do exist. There are two, on two different topologies; over $\mathbb{Z}_{11}$ they are the path 0–1:5, 1–2:1, 2–3:2, 3–4:7 and the spider 0–1:1, 1–2:5, 0–3:3, 0–4:7, with 4 unit-orbits in total. The ten path sums of the first are $5, 1, 2, 7, 6, 3, 9, 8, 10$ and $15 \equiv 4$, and those of the second are $1, 5, 6, 3, 7, 4, 8, 9, 13 \equiv 2$ and 10: in both cases exactly $\mathbb{Z}_{11} \setminus \{0\}$, as the reader can check in a minute. Both labellings use distinct nonzero labels, so they also satisfy the more restrictive readings of the definition. We found these by exhaustive search over all 10^4 labellings of all three topologies as well as by both search engines. We therefore report a discrepancy with the claim quoted above, without being able to locate its source: it may be a difference of definition in [10], which we have not read, or an error in that paper or in its summary. Everything else we compute agrees with [9].

6.3 Leaf-Leech trees: six leaves, and Question 3.1 of Ozen–Wang–Yalman

Ozen, Wang and Yalman [15] define: "A (not weighted) tree T with n leaves is a leaf-Leech tree if the distances between pairs of leaves are exactly $3, 4, \ldots, \binom{n}{2}+2$" [15, Def. 1] (distances 1 and 2 between leaves are impossible except in trivial cases, whence the shift). They prove the Taylor analogue (the number of leaves is k^2 or k^2+2), that the *expansion* T^e of a Leech tree—subdivide each edge into a path of its weight and hang a pendant vertex at every original vertex—is leaf-Leech, and, conversely, that a leaf-Leech tree with no *irreducible* vertex (a vertex "with degree at least three and no leaf neighbors" [15, p. 4]) is the expansion of a Leech tree. They then ask [15, Question 3.1]: "Does there exist a leaf-Leech tree that contains an irreducible vertex?", remarking that they have not been able to find a leaf-Leech tree that is not the expansion of a Leech tree; before the present note the largest number of leaves for which any leaf-Leech tree was known was 6, from Leech's order-6 tree.

Contracting the degree-2 vertices turns a leaf-Leech tree with L leaves into a series-reduced tree (all internal degrees ≥ 3) with L leaves and positive integer edge weights whose $S = \binom{L}{2}$ weighted leaf-to-leaf distances are exactly $\{3, \ldots, S+2\}$; conversely every such weighted tree expands to a leaf-Leech tree. The series-reduced topologies ($L+1 \leq |V| \leq 2L-2$) were enumerated with nauty's `gentreeg` and cross-checked against NetworkX, and each was solved with CP-SAT [16] in enumerate-all-solutions mode (weights in $[1, S+2]$, an `AllDifferent` over the S leaf-pair distance variables constrained to $[3, S+2]$, which forces the bijection); solutions were de-duplicated modulo isomorphism of the expanded unweighted tree and re-checked by the independent checker.

Proposition 6.2. *There are exactly six leaf-Leech trees with six leaves. Exactly one of them, the expansion of Leech's order-6 tree, has no irreducible vertex; the other five have one. In particular the answer to Question 3.1 of [15] is yes.*

For $L = 3$ and $L = 4$ the enumeration gives 1 and 2 leaf-Leech trees, namely the expansions of the Leech trees of those orders, and for $L = 5$ none (as Proposition 1 of [15] already excludes). That exactly one of the six trees at $L = 6$ is irreducible-vertex-free is a consistency check on the enumeration, since by their Theorem 2 such a tree must be the expansion of a Leech tree of order 6, and there is exactly one of those. An explicit witness for Proposition 6.2, in contracted form, is

$$1\text{–}0{:}1,\ 1\text{–}2{:}6,\ 1\text{–}5{:}4,\ 0\text{–}6{:}1,\ 0\text{–}9{:}1,\ 2\text{–}3{:}6,\ 2\text{–}4{:}2,\ 6\text{–}7{:}3,\ 6\text{–}8{:}1,$$

whose expansion has 26 vertices and six leaves $3, 4, 5, 7, 8, 9$ at the fifteen pairwise distances $3, 4, \ldots, 17$; the vertices 1 and 2 have degree 3 and, in the expanded tree, no leaf neighbour, so both are irreducible. This tree is therefore not the expansion of any Leech tree.

The next admissible numbers of leaves are $L = 9$ and $L = 11$ (73 and 488 series-reduced topologies), where CP-SAT is no longer adequate—exactly as in the Leech problem itself, it decides the small cases and then stalls (the 10-vertex topology at $L = 9$ is infeasible in 41 s, the 11-vertex ones were undecided after 3600 s). A forcing-style engine over leaf distances, analogous to Section 3, would be the natural next step; we leave it open.

6.4 Status of the results of this section

These are computational results of the same kind as Theorem 1.1, with weaker but explicitly stated verification. The existence statements (the two minimal trees at $n = 11$, the $D = 78$ tree at $n = 12$, the modular trees of order 5, the six leaf-Leech trees) are witnesses, printed above or in the archive, and are checkable by hand or by any independent checker; they do not depend on the search being exhaustive. The non-existence statements ($M(11) \geq 60$, $M(12) \geq 77$, no modular Leech tree of order 9 or 11, only six leaf-Leech trees at $L = 6$) do

depend on exhaustiveness: $M(11) \geq 60$ and the count of minimal trees at $n = 11$ are confirmed by a second, structurally different program on two architectures; the modular order-9 result by two implementations with different symmetry breaking; the modular order-11 result by one implementation only; the leaf-Leech enumeration by a single CP-SAT model over an independently cross-checked topology list. We label them accordingly and regard the modular order-11 result as the least independently supported of the group.

7 Discussion

Status of the result. Theorem 1.1 is a computer-assisted theorem in the usual sense: the mathematics (Lemmas 2.1, 3.1, 3.2 and Proposition 3.3) is short and has been checked by hand and by exhaustive small cases; the remainder is a finite computation whose correctness depends on the faithful execution of a program. What distinguishes this computation from a bare "no solution found" is that its intermediate output—the eighteen per-level counts of Table 4—is a mathematically defined sequence (the number of non-isomorphic forced forests of each size), so that any independent implementation can be checked against it level by level. The shallow levels have been confirmed by an oracle written from the definitions; all eighteen levels have been reproduced by an independent implementation written from the algorithm description; and three replications of the reference engine on different hardware and compilers agree bit-for-bit. The one verification we would like to have and do not yet have in full is the independent *method*: the per-topology engine, which shares no code and no search order with the forest engine, has decided 73,021 of the 122,344 surviving 18-vertex topologies (all UNSAT, none SAT) and is still running on the remainder (Section 4.2). It has, however, already reproduced the complete order-16 result (19,320 topologies, all UNSAT) by this independent route, which is the strongest available calibration of the two engines against each other and against [3].

Certificates. We do not have a proof certificate for $n = 18$ that could be checked by a formally verified checker, as we do for $n \leq 11$ (DRAT). Two routes are open. (i) Cube-and-conquer: use the forcing-lemma tree to a fixed depth as cubes and certify each cube with a CDCL solver producing DRAT proofs; the completeness of the cube set follows from Lemma 2.1 by a one-paragraph argument, so the search engine's pruning need not be trusted, only its branching order. Our experiments at $n = 16$ suggest a cost of the order of one CPU-hour per topology and proofs of 10^2 MB per cube, i.e. 10^4–10^5 CPU-hours for all of $n = 18$—feasible on a cluster but not done. (ii) Proof logging inside the forest engine (VeriPB/pseudo-Boolean), which would certify the isomorph rejection as well; not attempted. We regard the present level of verification (independent oracle for the shallow levels, differential tests on planted instances, three bit-identical replications, and an independent implementation reproducing all per-level counts) as adequate for the claim, and the per-level counts as the reproducible artefact that a sceptical reader should target.

What remains. The next admissible orders are $n = 25$ ($N = 300$) and $n = 27$ ($N = 351$). Extrapolating the growth factor of 6.5–7 per unit of n gives of the order of 10^{16}–10^{17} nodes at $n = 25$, which is beyond the present method by four to five orders of magnitude; new pruning ideas (for instance exploiting the top-value structure and the parity classes inside the forest search, or a meet-in-the-middle over the smallest and largest weights) or a genuinely different argument would be needed. Whether Leech's five trees are the only Leech trees remains open; the present note only moves the smallest open order from 18 to 25. In the neighbouring problems of Section 6 the frontier is much closer: the exact value of $M(12)$ needs one exhaustive run that is queued, $M(13)$ is out of reach of the present engine, modular

Leech trees of order 12 and above are untouched, and the leaf-Leech question at 9 leaves needs an engine of the kind built here for Leech trees rather than a generic solver.

8 AI-assisted research statement

The search programs, the test and verification harnesses, the literature ledger and the drafts of this note were produced by AI coding agents (Claude, Anthropic, in the Claude Code environment) working under the direction of the author, who set the problem, chose the algorithmic route (forced-forest search with isomorph rejection), decided what had to be verified, ran the cluster jobs and reviewed the proofs and the code. An adversarial "referee" pass over both engines was likewise carried out by an AI agent instructed to look for unsound pruning and incomplete symmetry breaking; it produced the correction to Lemma 2.5 recorded in Section 2 and the verification scheme of Section 4. The clean-room implementation of Section 4.5 was written by a separate agent that had no access to the production source. All mathematical claims in this note were checked by independent implementations as described in Section 4; human review of the proofs, the code and the text is ongoing, and the author takes responsibility for the content. **[TODO: author to confirm and adjust this statement]**

9 Data and code availability

All code and data are available at https://github.com/lsmeng/leech-trees **[TODO: repository to be created; add archival DOI (Zenodo) at submission]**. The archive contains: the two search engines (`forest_search.cpp`, `leech_search.cpp`) and the clean-room `cr_forest.c`; the independent Python enumerators and differential harnesses; the SAT encoder and the DRAT-verified proofs for $n \leq 11$; the frozen list of the 123,867 trees of order 18 with the two-generator cross-check; the per-shard output records and depth histograms of the $n = 18$ runs and of the $n \leq 17$ runs; the verdict scripts; and the referee reports on which Section 4 is based. It also contains, in a separate directory, the modified engines and the raw outputs behind Section 6 (minimal distinct distance trees, modular Leech trees, leaf-Leech trees), including the witnesses printed above and the independent checker used on them. The five known Leech trees are checked by two independent checkers, and every witness produced by any engine in the project was passed through both.

Acknowledgements

Computations were carried out on the author's laptop and on the Hoffman2 Shared Cluster of the UCLA Office of Advanced Research Computing. **[TODO: confirm acknowledgement wording; add funding if any]**

References

[1] A. Biere, K. Fazekas, M. Fleury, and M. Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020. In T. Balyo, N. Froleyks, M. Heule, M. Iser, M. Järvisalo, and M. Suda, editors, *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*, volume B-2020-1 of *Department of Computer Science Report Series B*, pages 51–53. University of Helsinki, 2020.

[2] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt. CaDiCaL 2.0. In *Computer Aided Verification (CAV 2024)*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024. doi: 10.1007/978-3-031-65627-9_7.

[3] B. Calhoun, K. Ferland, L. Lister, and J. Polhill. Minimal distinct distance trees. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 61:33–57, 2007.

[4] distributed.net. OGR (optimal Golomb rulers) project. https://www.distributed.net/OGR, 2022. Distributed verification of optimal Golomb rulers with 24–28 marks (OGR-24 to OGR-28, 2004–2022). Accessed 2026-08-18.

[5] A. Dollas, W. T. Rankin, and D. McCracken. A new algorithm for Golomb ruler derivation and proof of the 19 mark ruler. *IEEE Transactions on Information Theory*, 44(1):379–382, 1998. doi: 10.1109/18.651068.

[6] Epoch AI. FrontierMath open problems: Leech trees. https://epoch.ai/frontiermath/open-problems/leech-trees, 2025. Accessed 2026-08-18.

[7] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.

[8] A. Lakshmanan S. and M. M. Eldho. On Leech labelings of graphs and some related concepts. *Discrete Mathematics*, 347(4):113837, 2024. doi: 10.1016/j.disc.2023.113837.

[9] D. Leach. Modular Leech trees of order at most 8. *International Journal of Combinatorics*, 2014:Article ID 218086, 2 pages, 2014. doi: 10.1155/2014/218086.

[10] D. Leach and M. Walsh. Generalized Leech trees. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 78:15–22, 2011. Not consulted by the present author; cited as the source of a statement quoted in Leach (2014).

[11] J. Leech. Another tree labelling problem. *The American Mathematical Monthly*, 82(9):923–925, 1975. doi: 10.1080/00029890.1975.11993981.

[12] T. Luo and L. Yu. A graph labeling problem. *Involve, a Journal of Mathematics*, 17(2):327–335, 2024. doi: 10.2140/involve.2024.17.327.

[13] B. D. McKay and A. Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014. doi: 10.1016/j.jsc.2013.09.003.

[14] OEIS Foundation Inc. The on-line encyclopedia of integer sequences. https://oeis.org, 2026. Sequences A000055 (number of trees with n unlabeled nodes) and A003022 (length of optimal Golomb rulers). Accessed 2026-08-18.

[15] M. Ozen, H. Wang, and D. Yalman. Note on Leech-type questions of trees. *Integers*, 16:Paper No. A21, 2016.

[16] L. Perron and F. Didier. CP-SAT, OR-Tools v9. Google, https://developers.google.com/optimization/cp/cp_solver, 2024. Accessed 2026-08-18.

[17] J. B. Shearer. Some new optimum Golomb rulers. *IEEE Transactions on Information Theory*, 36(1):183–184, 1990. doi: 10.1109/18.50388.

[18] L. A. Székely, H. Wang, and Y. Zhang. Some non-existence results on Leech trees. *Bulletin of the Institute of Combinatorics and its Applications*, 44:37–45, 2005.

[19] H. Taylor. Odd path sums in an edge-labeled tree. *Mathematics Magazine*, 50(5):258–259, 1977. doi: 10.1080/0025570X.1977.11976658.

[20] H. Taylor. A distinct distance set of 9 nodes in a tree of diameter 36. *Discrete Mathematics*, 93(2–3):167–168, 1991. doi: 10.1016/0012-365X(91)90252-W.

[21] S. Varghese, A. Lakshmanan S., and S. Arumugam. Two classes of non-Leech trees. *Electronic Journal of Graph Theory and Applications*, 8(1):205–210, 2020. doi: 10.5614/ejgta.2020.8.1.15.

[22] N. Wetzler, M. J. H. Heule, and W. A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 422–429. Springer, 2014. doi: 10.1007/978-3-319-09284-3_31.

[23] R. A. Wright, B. Richmond, A. Odlyzko, and B. D. McKay. Constant time generation of free trees. *SIAM Journal on Computing*, 15(2):540–548, 1986. doi: 10.1137/0215039.